

Review of ImageNet Classification with Deep Convolutional Neural Networks

Abhishu Panthi
Tribhuwan University
080bct009.abhishu@pcampus.edu.np

Ishan Gautam
Tribhuwan University
080bct034.ishan@pcampus.edu.np

Kushal K.C.
Tribhuwan University
080bct041.kushal@pcampus.edu.np

Abstract—In this paper, the contributions of Krizhevsky, Sutskever, and Hinton on ImageNet classification via deep convolutional neural networks, that scored 15.3 on ILSVRC-2012. We will analyze the architecture of the Activations for faster convergence, and multi-GPU parallelization. The most important regularization technique is dropout regularization on fully-connected layers and extensive data augmentation using Random cropping, horizontal flipping, and PCA-based color transformation perturbation. The training procedure relied on stochastic gradient descent along with momentum, weight decay, and adaptive learning rate scheduling. We examine particular design decisions, including Local Response Normalization, Overlapping Pooling, Weight Initialization strategies. This paper will cover learned feature representations, demonstrating GPU specialization and semantic abstraction in higher layers. We place the AlexNet architecture in the context of subsequent developments, analyzing its shallow depth, large 11×11 kernels, and LRN and their evolution into batch normalization, smaller filters, and ResNet architectures. **Index Terms** - Deep Learning, Convolutional Neural Networks, ImageNet, AlexNet, Computer Vision, Image Classification, ReLU, Drop-out, GPU Computing

Index Terms—Deep Learning, Convolutional Neural Networks, ImageNet, AlexNet, Computer Vision, Image Classification, ReLU, Dropout, GPU Computing

I. INTRODUCTION

Deep learning techniques have revolutionized computer vision. For example, convolutional neural networks (CNNs) - The Rising Stars criterion for image recognition problems. In the pioneering works in this domain, the paper “ImageNet Classification “with Deep Convolutional Neural Networks” by Krizhevsky Sutskever, and Hinton [1] was a pivotal event that triggered the development of the “deep learning revolution.”

A. Historical Context

Prior to 2012, computer vision systems primarily relied on hand-crafted features combined with traditional machine learning algorithms. Methods such as SIFT (Scale-Invariant Feature Transform) combined with Fisher vectors achieved a top-5 error rate of 26.2% on the ILSVRC-2012 competition [4]. The limitation of these approaches was the inability to scale with the increasing complexity of visual data. These methods were powerful but were limited by a lack of adaptability.

II. NETWORK ARCHITECTURE

A. Overall Design

AlexNet consists of eight learned layers: five convolutional layers followed by three fully-connected layers. The network processes $224 \times 224 \times 3$ RGB images and the output of the last fully-connected layer is fed to a 1000-way softmax which produces a distribution over 1000 class labels, with approximately 60 million parameters.

B. Convolutional Layers

The initial layer uses 96 kernels sized $11 \times 11 \times 3$, which convolve with the input image using a stride of 4 pixels. This large stride helps reduce the output spatial volume size significantly. This layer is responsible for capturing low-level features such as oriented edges and color blobs, which are then passed through ReLU activation, Local Response Normalization (LRN), and max-pooling. The subsequent layers of the network refine these initial features into increasingly abstract representations. The second convolution layer uses 256 kernels of size $5 \times 5 \times 48$. The use of 48 feature maps corresponds to separate filtered outputs from the previous layer. Layers 3 and 4 are then connected directly without pooling and normalization layers. The third layer includes 384 kernels of size $3 \times 3 \times 256$; the fourth mirrors the third layer’s configuration without altering the layer’s spatial dimensions using 384 kernels of size $3 \times 3 \times 192$. The final convolutional layer uses 256 kernels of size $3 \times 3 \times 192$ followed by a max-pooling layer which reduces dimensionality as well as rotational and positional invariance to the features being learned.

C. Fully-Connected Layers

The first and second fully connected layers are dense layers of 4096 neurons; they take flattened output from preceding convolution layers and project onto high-dimensional space to learn non-linear combinations of these features. The final fully connected layer has as many neurons as there are classes (1000), and utilizes a softmax function to derive the classification probabilities. The softmax not only outputs these probabilities but also forms the basis for cross-entropy loss during training.

D. Multi-GPU Architecture

A distinctive feature of AlexNet is its parallelization across two GPUs. The network distributes kernels across both GPUs, with inter-GPU communication occurring only at specific layers. For instance, layer 3 kernels receive inputs from all kernel maps in layer 2, whereas layer 4 kernels only receive inputs from layer 3 on the same GPU. This design choice was needed primarily because of memory constraints while enabling the training of a network of unprecedented size.

III. ALEXNET COMPONENTS

A. ReLU Nonlinearity

The most impactful contribution of AlexNet was demonstrating the effectiveness of Rectified Linear Units (ReLU) as activation functions [2]. The ReLU function is defined as:

$$f(x) = \max(0, x) \quad (1)$$

Unlike traditional saturating nonlinearities such as $\tanh(x)$ or sigmoid functions $\sigma(x) = (1 + e^{-x})^{-1}$, ReLUs help networks converge faster by addressing the vanishing gradient problem. The gradient of the ReLU function is either 0 for negative inputs or 1 for positive inputs, which speeds up gradient descent. It promotes sparsity of activation. Since it outputs zero for half of its input domain, it inherently produces sparse data representations. The authors demonstrated that networks with ReLUs train several times faster than equivalent networks with saturating activation functions, reaching 25% training error on CIFAR-10 approximately six times faster than networks using $\tanh$ units.

B. Local Response Normalization

Although ReLUs do not require input normalization to prevent saturation, AlexNet uses local response normalization (LRN) to aid generalization. For a neuron with activity $a_{x,y}^i$ at position (x, y) using kernel i , the normalized activity $b_{x,y}^i$ is computed as:

$$b_{x,y}^i = a_{x,y}^i / \left(k + \alpha \sum_{j=\max(0, i-n/2)}^{\min(N-1, i+n/2)} (a_{x,y}^j)^2 \right)^\beta \quad (2)$$

where the summation runs over n adjacent kernel maps at the same spatial position, N is the total number of kernels in the layer, and k , α , β , and n are hyperparameters. The authors used $k = 2$, $n = 5$, $\alpha = 10^{-4}$, and $\beta = 0.75$.

LRN implements a form of lateral inhibition, a biological process where activated neurons suppress the activity of neighboring neurons in the same layer, creating competition among neuron outputs computed using different kernels. This inhibition contributes to activity regulation by preventing any single feature map from overwhelming the response of the network by penalizing larger activations that lack support from their surroundings. By focusing on patterns that stand out relative to their neighbors, LRN functions similarly to contrast normalization. Incorporating LRN reduced the top-1 and top-5 error rates by 1.4% and 1.2%, respectively.

C. Overlapping Pooling

AlexNet applies overlapping pooling with $s = 2$ and $z = 3$, meaning adjacent pooling windows overlap by one pixel, reducing top-1 and top-5 error rates by 0.4% and 0.3% respectively. This increases the density of feature mapping while making the model slightly more resistant to overfitting.

IV. REGULARIZATION TECHNIQUES

A. Data Augmentation

Given the massive parameter count relative to available training data, overfitting posed a significant challenge. AlexNet addresses this through data augmentation:

1) *Image Translations and Reflections*: The network extracts random 224×224 patches from 256×256 images during training, along with their horizontal reflections. This increased the training set size by a factor of 2,048. At test time, the network averages predictions over ten patches (four corners, center, and their horizontal reflections) for more stable results.

2) *PCA-based Color Augmentation*: For each training image, the method adds multiples of principal components to RGB values:

$$[p_1, p_2, p_3][\alpha_1 \lambda_1, \alpha_2 \lambda_2, \alpha_3 \lambda_3]^T \quad (3)$$

where p_i and λ_i are the i -th eigenvector and eigenvalue of the RGB covariance matrix, and α_i are random variables drawn from a Gaussian distribution with mean zero and standard deviation 0.1. This captures the invariance of object identity to changes in illumination intensity and color, reducing top-1 error rate by over 1%.

B. Dropout

Dropout regularization randomly sets the output of hidden neurons to zero with probability 0.5 during training [3]. This prevents complex co-adaptations of neurons, as each neuron must learn robust features useful in conjunction with many random subsets of other neurons.

At test time, all neurons are active but their outputs are multiplied by 0.5, approximating the geometric mean of predictions from the exponentially many dropout networks. Applied to the first two fully-connected layers, dropout roughly doubles the iterations required for convergence but substantially reduces overfitting.

V. TRAINING METHODOLOGY AND OPTIMIZATION

A. Optimization

The network was trained using stochastic gradient descent (SGD) with the following hyperparameters:

- Batch size: 128 examples
- Momentum: 0.9
- Weight decay: 0.0005
- Initial learning rate: 0.01

The weight update rule follows:

$$v_{i+1} := 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \middle|_{w_i} \right\rangle_{D_i} \quad (4)$$

$$w_{i+1} := w_i + v_{i+1} \quad (5)$$

where i is the iteration index, v is the momentum variable, ϵ is the learning rate, and $\langle \partial L / \partial w |_{w_i} \rangle_{D_i}$ represents the average gradient over batch D_i . The batch size choice was a balance between computational efficiency and accuracy of estimating errors. While larger batch sizes provide a clearer signal for each update by reducing noise in gradient calculations, they also require more computational resources. A high level of momentum speeds up convergence towards the loss function's minimum, particularly useful when dealing with small but consistent gradients. Weight decay acts as a regularization term that penalizes large weights by adding a portion of weight values to the loss function; this ensures that the model does not rely too heavily on a small number of high-weight features. The learning rate is decreased by a factor of 10 at predetermined points during training, also known as "step decay"; these adjustments typically occur when the validation error rate stops decreasing significantly. Reducing it helps refine weight adjustments, promoting better convergence.

B. Initialization

Weights were initialized from a zero-mean Gaussian distribution with standard deviation 0.01; a narrow standard deviation prevents any single neuron from initially overwhelming the output. Neuron biases in the second, fourth, fifth convolutional layers and fully-connected layers were initialized to 1.0 to provide ReLUs with positive inputs initially. Remaining layer biases were initialized to 0.

VI. RESULTS AND PERFORMANCE

A. ILSVRC-2010 Results

On the ILSVRC-2010 test set, AlexNet achieved top-1 and top-5 error rates of 37.5% and 17.0% respectively. This represented a substantial improvement over previous methods:

- Sparse coding approach: 47.1% top-1, 28.2% top-5
- SIFT + Fisher Vectors: 45.7% top-1, 25.7% top-5

B. ILSVRC-2012 Competition

In the ILSVRC-2012 competition:

- Single CNN: 18.2% top-5 error
- Ensemble of 5 CNNs: 16.4% top-5 error
- CNN pre-trained on Fall 2011 release: 16.6% top-5 error
- Ensemble of 7 CNNs: 15.3% top-5 error (winning entry)

The second-place entry achieved 26.2% top-5 error.

C. Ablation Studies

Removing any single convolutional layer resulted in approximately 2% degradation in top-1 performance, despite each convolutional layer containing less than 1% of the model's parameters, emphasizing that depth itself is a critical factor in the network's representational capacity.

VII. ANALYSIS AND DISCUSSION

A. Learned Features

Qualitative analysis of learned convolutional kernels reveals interpretable patterns. The first layer learns frequency-selective and orientation-selective filters resembling Gabor filters, along with color blob detectors. Interestingly, the two GPUs exhibit specialization: GPU-1 learns largely color-agnostic features while GPU-2 learns color-specific features.

Feature similarity analysis using Euclidean distance in the 4,096-dimensional space of the last hidden layer shows that the network learns semantically meaningful representations. The model successfully abstracts high-level concepts, as evidenced by retrieved nearest neighbors often sharing semantic similarity despite substantial pixel-level differences.

B. Evolution

While AlexNet represented a monumental leap in performance, its architectural and methodological choices have been rapidly refined by subsequent research. These limitations are not failures but rather initial design points that were optimized as the field matured.

- 1) **Depth:** The eight-layer architecture represented the practical limit of trainable depth in 2012, but it constrained the network's representational capacity for hierarchical feature learning. AlexNet is relatively shallow compared to subsequent architectures like VGGNet (19 layers) [5], GoogLeNet (22 layers) [6], and ResNet (up to 152 layers) [7].
- 2) **Large filters:** The 11×11 kernels in the first layer are computationally expensive; each kernel contains 121 parameters per channel. For the first layer with 96 filters and 3 input channels, this resulted in 34,848 parameters for the first layer alone. Evolution to smaller filters followed the insight that n stacked 3×3 convolutions could yield an effective receptive field of $(2n+1) \times (2n+1)$ while requiring only $9n$ parameters versus $(2n+1)^2$ parameters for a single large convolutional filter. VGGNet formalized this by employing homogeneous stacks of 3×3 convolutions, while Inception architectures combined multiple filter sizes in parallel branches, allowing multi-scale feature extraction within a single layer.
- 3) **Local Response Normalization:** Local Response Normalization (LRN) was introduced as a form of lateral inhibition but addressed neither covariate shift nor gradient flow stabilization and relied on empirically chosen (k, α, β, n) . Batch Normalization (BN) provided a solution to internal covariate shift by changing the distribution of layer inputs during training [8]. BN normalizes activations within each mini-batch as follows:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2, \quad (6)$$

$$\hat{y}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{y}_i + \beta, \quad (7)$$

where γ and β are learnable parameters that restore the representational capacity of the network. By stabilizing activation distributions, it allows for higher learning rates and accelerates convergence. Additionally, the stochasticity introduced by mini-batch statistics provides implicit regularization, reducing overfitting and sensitivity to initialization.

- 4) **Weight initialization:** Gaussian initialization with $W \sim \mathcal{N}(0, 0.01^2)$ ignores signal propagation in deep networks. This causes exponential growth or decay of activation variances across layers. Newer approaches such as Xavier initialization [9] preserve forward-pass variance:

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \quad (8)$$

VIII. CONCLUSION

AlexNet has been a pivotal moment for neural network design and training techniques. AlexNet’s success in 2012 was enabled by large-scale datasets, GPU computing, and improved training methods. The paper’s technical contributions—ReLU activation, dropout regularization, GPU acceleration, and effective data augmentation—have become fundamental tools. More broadly, AlexNet validated end-to-end learning and verified that with sufficient data and computation, neural networks could learn rich hierarchical representations. While subsequent architectures have improved upon AlexNet’s performance through increased depth, sophisticated normalization techniques, and connectivity patterns, the core insights remain relevant.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems* 25, 2012, pp. 1097–1105.
- [2] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 2010, pp. 807–814.
- [3] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, “Improving neural networks by preventing co-adaptation of feature detectors,” *arXiv preprint arXiv:1207.0580*, 2012.
- [4] O. Russakovsky *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.
- [5] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [6] C. Szegedy *et al.*, “Going deeper with convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [8] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating deep network training by reducing internal covariate shift,” in *International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [9] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010, pp. 249–256.